

Regular Expressions (RegExp) in JavaScript

Regular Expression বা **RegExp** হলো একটি pattern language, যার মাধ্যমে string-এর মধ্যে নির্দিষ্ট pattern **search**, **match**, **extract**, **validate**, **replace**, বা **split** করা যায়।



Topics Covered

1. RegExp Literals & `RegExp` Constructor
2. `test()`, `match()`, `matchAll()`, `replace()`, `search()`, `split()`
3. Character Classes, Quantifiers & Groups
4. Lookahead & Lookbehind Assertions
5. Named Capture Groups
6. Unicode-aware Regex with `/u` Flag

Related concepts covered:

- Regex flags
 - Escaping
 - Anchors
 - Capturing vs non-capturing groups
 - Greedy vs lazy quantifiers
 - Backreferences
 - Alternation
 - Word boundaries
 - Practical validation patterns
 - Common RegExp mistakes
 - Performance considerations
-

1. What is Regular Expression?

ধরো:

```
const text = "My phone number is 01712345678";
```

আমরা যদি phone number খুঁজতে চাই, manually character-by-character check করতে পারি।

কিন্তু RegExp দিয়ে pattern define করতে পারি:

```
const pattern = /\d+/;
```

এখানে:

```
\d  
↓  
digit  
  
+  
↓  
one or more
```

তাই:

```
/\d+/
```

মানে:

এক বা একাধিক digit-এর sequence খুঁজো।

2. কেন **Regex** ব্যবহার করা হয়?

Common use cases:

- Email validation
- Phone number validation
- Password rules
- Username validation
- URL extraction
- Search
- Text replacement
- Log parsing
- Data extraction
- Input sanitization
- String transformation

3. Regex-এর **Basic Structure**

একটি regex:

```
/pattern/flags
```

Example:

```
/hello/i
```

এখানে:

```
/hello/  
  ↓  
pattern  
  
i  
  ↓  
flag
```

4. Regex Literal

JavaScript-এ সবচেয়ে সহজ way:

```
const regex = /hello/;
```

এটাকে **regex literal** বলা হয়।

Example:

```
const regex = /javascript/i;  
  
console.log(regex.test("I love JavaScript"));
```

Output:

```
true
```

5. RegExp Constructor

Regex object constructor দিয়েও তৈরি করা যায়:

```
const regex = new RegExp("hello");
```

Use:

```
console.log(  
  regex.test("hello world")  
);
```

6. Literal vs Constructor

Literal

```
const regex = /hello/i;
```

Constructor

```
const regex = new RegExp("hello", "i");
```

দুটো conceptually equivalent ।

7. কখন RegExp Constructor দরকার?

Dynamic pattern তৈরি করতে constructor useful ।

```
const word = "javascript";  
  
const regex = new RegExp(word, "i");  
  
console.log(  
  regex.test("I love JavaScript")  
);
```

এখানে pattern variable থেকে এসেছে।

8. Dynamic Regex

```
const username = "shariar";

const regex = new RegExp(`^${username}$`);

console.log(
  regex.test("shariar")
);
```

কিন্তু dynamic input সরাসরি regex-এ বসানোর সময় escaping/security considerations মাথায় রাখতে হবে।

9. Regex Flags

Common flags:

Flag	Meaning
<code>g</code>	Global
<code>i</code>	Case-insensitive
<code>m</code>	Multiline
<code>s</code>	Dot matches newline
<code>u</code>	Unicode-aware
<code>y</code>	Sticky
<code>d</code>	Match indices

আমাদের এই section-এ বিশেষভাবে `/u` গুরুত্বপূর্ণ।

10. `i` Flag

Without `i`:

```
/hello/.test("Hello");
```

Output:

```
false
```

With `i`:

```
/hello/i.test("Hello");
```

Output:

```
true
```

11. `g` Flag

`g` মানে global matching।

```
const regex = /cat/g;  
const text = "cat dog cat cat";
```

Global match:

```
text.match(regex);
```

Result:

```
["cat", "cat", "cat"]
```

12. Important: `/g` এবং `test()`

`g` flag থাকলে `test()` stateful behavior দেখাতে পারে কারণ regex-এর `lastIndex` পরিবর্তিত হয়।

Example:

```
const regex = /cat/g;

console.log(regex.test("cat cat"));
console.log(regex.test("cat cat"));
console.log(regex.lastIndex);
```

তাই repeated `test()` করার সময় global regex-এর state সম্পর্কে সতর্ক থাকতে হবে।

13. Escaping

Regex-এ কিছু character special meaning বহন করে।

যেমন:

```
.
*
+
?
(
)
[
]
{
}
^
$
|
```

Literal special character match করতে escape করতে হয়।

Example:

```
/\./
```

এখানে `.`-কে literal dot হিসেবে match করা হচ্ছে।

14. Dot .

Regex-এ:

```
/./
```

এর অর্থ সাধারণত:

একটি character

Example:

```
/a.c/
```

match করতে পারে:

```
abc  
a-c  
a c  
a9c
```

কারণ middle-এ যেকোনো single character থাকতে পারে।

তবে defaultভাবে newline match করে না।

`/s` flag ব্যবহার করলে dot newline-ও match করতে পারে।

15. test()

`test()` check করে pattern string-এর মধ্যে match করে কিনা।

```
const regex = /hello/;  
  
console.log(  
  regex.test("hello world")  
);
```

Output:

```
true
```

No match:

```
console.log(  
  regex.test("goodbye world")  
);
```

Output:

```
false
```

16. match()

String-এর `match()` method regex দিয়ে match খুঁজে।

```
const text = "I have 10 apples and 20 oranges.";  
  
const result = text.match(/\d+/);  
  
console.log(result);
```

Without `g`, first match-এর details পাওয়া যায়।

With `g`

```
const result = text.match(/\d+/g);  
  
console.log(result);
```

Result:

```
["10", "20"]
```

17. `matchAll()`

`matchAll()` সব matches-এর detailed iterator দেয়।

```
const text = "ID: 101, ID: 202";

const matches = text.matchAll(/\d+/g);

for (const match of matches) {
  console.log(match[0]);
}
```

Output:

```
101
202
```

18. `match()` vs `matchAll()`

	<code>match()</code>	<code>matchAll()</code>
Return	Array / null	Iterator
Global matches	Values পাওয়া যায়	Detailed match records
Capture groups	Global mode-এ limited	প্রতিটি match-এর groups
Iteration	Direct	<code>for...of</code> / spread
Requires <code>g</code> for all matches	Yes	Yes

19. Capture Groups with `matchAll()`

```
const text = "John: 25, Sara: 30";

const regex = /(\w+): (\d+)/g;

for (const match of text.matchAll(regex)) {
  console.log(match[1]);
}
```

```
    console.log(match[2]);  
  }
```

Output conceptually:

```
John  
25  
  
Sara  
30
```

20. `replace()`

String-এর matching অংশ replace করতে:

```
const text = "I like cats.";

const result = text.replace(
  "cats",
  "dogs"
);

console.log(result);
```

Output:

```
I like dogs.
```

21. `replace()` with Regex

```
const text = "cat cat cat";

const result = text.replace(
  /cat/g,
  "dog"
);
```

```
console.log(result);
```

Output:

```
dog dog dog
```

22. `replace()` with Function

Replacement হিসেবে function দেওয়া যায়।

```
const text = "10 20 30";  
  
const result = text.replace(  
  /\d+/g,  
  match => Number(match) * 2  
);  
  
console.log(result);
```

Output:

```
20 40 60
```

23. `search()`

`search()` প্রথম matching position/index return করে।

```
const text = "Hello JavaScript";  
  
const index = text.search(/JavaScript/);  
  
console.log(index);
```

Output:

6

Match না পেলো:

-1

24. `search()` vs `indexOf()`

```
text.indexOf("JavaScript");
```

শুধু literal substring-এর জন্য।

আর:

```
text.search(/java(script)?/i);
```

pattern matching করতে পারে।

25. `split()`

Regex ব্যবহার করে string split করা যায়।

```
const text = "apple,banana;orange";  
const result = text.split(/[,;]/);  
console.log(result);
```

Output:

```
["apple", "banana", "orange"]
```

26. Regex Methods Overview

```
Regex
├── regex.test(string)
│   └── true / false
└── string methods
    ├── match()
    ├── matchAll()
    ├── replace()
    ├── search()
    └── split()
```

27. Character Classes

Character class হলো এমন pattern যা নির্দিষ্ট set-এর একটি character match করে।

[abc]

`/[abc]/`

মানে:

`a`, `b`, অথবা `c` -এর যেকোনো একটি।

Example:

`/[abc]/.test("cat");`

Output:

true

28. Range

```
/[a-z]/
```

মানে:

a থেকে z

Similarly:

```
/[A-Z]/
```

মানে:

A থেকে Z

Numbers:

```
/[0-9]/
```

29. Multiple Ranges

```
/[a-zA-Z0-9]/
```

মানে:

lowercase + uppercase + digit

30. Negated Character Class

^ class-এর শুরুতে থাকলে "not" বোঝায়।

```
/[^0-9]/
```

মানে:

digit ছাড়া যেকোনো character

Example:

```
/[^0-9]/.test("123a");
```

Output:

```
true
```

31. \d

```
/\d/
```

digit |

Equivalent concept:

```
/[0-9]/
```

32. \D

```
/\D/
```

non-digit |

33. \w

```
/\w/
```

সাধারণত ASCII word character:

A-Z
a-z
0-9
—

34. \W

/\W/

non-word character |

35. \s

Whitespace:

/\s/

যেমন:

- space
- tab
- newline

36. \S

Non-whitespace |

37. Character Class Cheat Sheet

Pattern	Meaning
[abc]	a/b/c
[^abc]	a/b/c ছাড়া

Pattern	Meaning
[a-z]	lowercase
[A-Z]	uppercase
[0-9]	digit
\d	digit
\D	non-digit
\w	ASCII word character
\W	non-word
\s	whitespace
\S	non-whitespace

38. Quantifiers

Quantifier বলে:

আগের pattern কতবার repeat হতে পারবে?

*

Zero or more |

/a*/

Matches:

" "
"a"
"aa"
"aaa"

+

One or more |

`/a+/`

Matches:

"a"
"aa"
"aaa"

কিছু empty string নয়।

?

Zero or one।

`/a?/`

39. {n}

Exactly `n` times।

`/\d{4}/`

চারটি digit।

Example:

2026

40. {n,}

At least `n` times।

`/\d{4,}/`

কমপক্ষে ৪টি digit ।

41. {n,m}

Between `n` and `m` ।

```
/\d{2,4}/
```

২ থেকে ৪টি digit ।

42. Quantifier Table

Quantifier	Meaning
<code>*</code>	0+
<code>+</code>	1+
<code>?</code>	0 or 1
<code>{n}</code>	Exactly n
<code>{n,}</code>	At least n
<code>{n,m}</code>	n to m

43. Greedy Quantifiers

Default quantifiers সাধারণত greedy ।

```
const text = "<b>Hello</b><b>World</b>";  
const result = text.match(/<b>.*</b>/);  
console.log(result[0]);
```

এটা বেশি text consume করতে পারে ।

44. Lazy Quantifiers

Lazy করতে `?` যোগ করা যায়।

```
const result = text.match(/<b>.*?<\b>/g);  
  
console.log(result);
```

Result:

```
[  
  "<b>Hello</b>",  
  "<b>World</b>"  
]
```

45. Anchors

`^`

String/line-এর beginning।

```
/^Hello/
```

মানে string `Hello` দিয়ে শুরু হতে হবে।

`$`

End।

```
/world$/
```

মানে `world` দিয়ে শেষ হতে হবে।

46. Full String Validation

Username পুরো string match করতে:

```
/^[a-zA-Z0-9_]+$
```

এখানে:

```
^  
↓  
start  
  
$  
↓  
end
```

তাই পুরো string pattern-এর মধ্যে থাকতে হবে।

47. Word Boundary \b

```
/\bcat\b/
```

এখানে `cat` একটি separate word হিসেবে match হবে।

এটা:

```
cat
```

match করবে।

কিন্তু:

```
catalog
```

match করবে না।

48. Groups

Parentheses দিয়ে group তৈরি করা যায়।

```
/(ab)+/
```

এখানে:

```
ab
```

একটি group।

49. Capturing Group

```
const regex = /(\d{4})-(\d{2})-(\d{2})/;
```

Example:

```
2026-09-03
```

Groups:

```
Group 1 → 2026  
Group 2 → 09  
Group 3 → 03
```

50. Alternation

OR operation।

```
/cat|dog/
```

মানে:

```
cat OR dog
```

Example:

```
/cat|dog/.test("I have a dog");
```

Output:

```
true
```

51. Group + Alternation

```
/(cat|dog)/
```

এখানে group-এর মধ্যে alternative ।

52. Non-Capturing Group

সব group-এর capture দরকার হয় না ।

তখন:

```
(?:...)
```

Example:

```
/(?:cat|dog)/
```

এটা group করবে কিন্তু capture করবে না ।

53. Capturing vs Non-Capturing

```
(cat|dog)
```

→ capturing

```
(?:cat|dog)
```

→ non-capturing

Non-capturing group useful যখন শুধু grouping দরকার, captured value দরকার নেই।

54. Backreferences

আগে capture করা group আবার reference করা যায়।

```
/(\w+)\s+\1/
```

এখানে:

```
(\w+)
```

একটি word capture করছে।

`\1` সেই same captured value reference করছে।

তাই:

```
hello hello
```

match করতে পারে।

কিন্তু:

```
hello world
```

match করবে না।

55. Lookahead Assertions

Lookahead pattern check করে সামনে নির্দিষ্ট কিছু আছে কিনা, কিন্তু সেটা consume করে না।

Positive Lookahead

Syntax:

```
(?=...)
```

Example:

```
/\d+(?=px)/
```

এটা এমন digit match করবে যার পরে `px` আছে।

Input:

```
100px
```

Match:

```
100
```

`px` match-এর অংশ হিসেবে consume হয়নি।

56. Negative Lookahead

Syntax:

```
(?!...)
```

মানে:

সামনে এই pattern থাকা যাবে না।

Example:

```
/\d+(?!px)/
```

এখানে digit-এর পরে `px` না থাকার condition।

57. Positive Lookahead Mental Model

```
100px
^^^
↓
\d+

      px
      ↓
lookahead checks
```

Lookahead:

"সামনে `px` আছে তো?"

কিন্তু `px` consume করে না।

58. Lookbehind Assertions

Lookbehind সামনে না দেখে পিছনে check করে।

Positive Lookbehind

Syntax:

```
(?<=...)
```

Example:

```
/ (?<= \$) \d+ /
```

এমন digits match করবে যার আগে \$ আছে।

Input:

\$100

Match:

100

\$ match-এর অংশ হবে না।

59. Negative Lookbehind

Syntax:

(?<!\...)

মানে:

পিছনে এই pattern থাকা যাবে না।

Example:

/(?<!\\$)\d+/?

60. Lookahead vs Lookbehind

Lookahead

↓

current position → future

Lookbehind

↑

past ← current position

Syntax:

(?=...)	positive lookahead
(?!...)	negative lookahead
(?<=...)	positive lookbehind
(?<!=...)	negative lookbehind

61. Password Validation Example

ধরো password-এর requirements:

- minimum 8 characters
- at least one digit
- at least one uppercase letter

Regex:

```
/^(?=.*[A-Z])(?=.*\d){8,}$/
```

Breakdown:

```
^
↓
start

(?=.*[A-Z])
↓
uppercase থাকতে হবে

(?=.*\d)
↓
digit থাকতে হবে

.{8,}
↓
minimum 8 characters

$
↓
end
```

62. Multiple Lookaheads

Lookahead একসাথে ব্যবহার করা যায়:

```
/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d){8,}$/
```

এখানে:

- uppercase
- lowercase
- digit
- minimum 8 characters

সব requirement।

63. Named Capture Groups

Traditional capture:

```
/(\d{4})-(\d{2})-(\d{2})/
```

এখানে:

```
\1  
\2  
\3
```

দিয়ে group access করতে হয়।

এটা less readable।

64. Named Group Syntax

Syntax:

```
(?<name>...)
```

Example:

```
const regex =  
  /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;
```

65. Named Group Example

```
const text = "2026-09-03";  
  
const regex =  
  /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;  
  
const match = text.match(regex);  
  
console.log(match.groups);
```

Result:

```
{  
  year: "2026",  
  month: "09",  
  day: "03"  
}
```

66. Why Named Groups Better?

Instead of:

```
match[1]  
match[2]  
match[3]
```

Use:

```
match.groups.year  
match.groups.month  
match.groups.day
```

Much more readable |

67. Named Groups with `matchAll()`

```
const text =  
  "2026-09-03 and 2027-10-15";  
  
const regex =  
  /(?:<year>\d{4}) - (?:<month>\d{2}) - (?:<day>\d{2})/g;  
  
for (const match of text.matchAll(regex)) {  
  console.log(match.groups.year);  
  console.log(match.groups.month);  
  console.log(match.groups.day);  
}
```

68. Named Backreference

Named group-কে name দিয়ে reference করা যায়:

```
\k<name>
```

Example:

```
/(?:<word>\w+)\s+\k<word>/
```

Match:

```
hello hello
```

69. Unicode

JavaScript strings Unicode-based ।

তাই শুধু ASCII character চিনলেই modern application-এর জন্য সবসময় যথেষ্ট নয় ।

Examples:

বাংলা
العربية
中文
日本語
é
😊

70. /u Flag

`u` flag regex-কে Unicode-aware mode-এ চালায় ।

```
const regex = /\p{L}+/u;
```

এখানে `\p{L}` Unicode letter property match করতে পারে ।

71. Unicode Property Escapes

`/u` ব্যবহার করে property escapes:

```
/\p{L}/u
```

→ Letter

```
/\p{N}/u
```

→ Number

```
/\p{Script=Bangla}/u
```

→ Bangla script characters

72. Unicode Letters Example

```
const regex = /^[\p{L}]+$/u;  
  
console.log(  
  regex.test("Shariar")  
);
```

```
true
```

বাংলা:

```
console.log(  
  regex.test("শরিয়ার")  
);
```

Unicode property-based matching-এর কারণে language-independent letter matching করা যায়।

73. Unicode Script

Bangla characters specifically:

```
const regex = /^[\p{Script=Bengali}]+$/u;
```

Example:

```
regex.test("বাংলা");
```

74. Unicode Numbers

```
const regex = /^p{N}+$/u;
```

এটি Unicode number characters-এর জন্য useful।

75. Emoji and Unicode

Unicode regex দিয়ে কিছু emoji-related properties detect করা যায়।

```
/\p{Emoji}/u
```

তবে emoji matching অনেক complex হতে পারে কারণ একটি visible emoji একাধিক Unicode code point/sequence দিয়ে তৈরি হতে পারে।

তাই:

Unicode-aware regex মানেই "সব emoji perfectly match করা" নয়।

76. `/u` কেন Important?

Without Unicode-aware handling, কিছু Unicode character sequence-এর behavior confusing হতে পারে।

`/u`:

- Unicode code points-এর interpretation improve করে
 - Unicode property escapes enable করে
 - modern international text processing-এর জন্য important
-

77. Regex + Internationalization

যদি application multilingual হয়:

```
English  
Bangla  
Arabic
```

Hindi
Chinese
Japanese
Emoji

তাহলে:

```
/[a-zA-Z]+/
```

এর উপর পুরোপুরি depend করা উচিত নয়।

Unicode-aware patterns consider করা দরকার।

78. Practical Email Pattern

একটি basic pattern:

```
const emailRegex =  
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Example:

```
emailRegex.test("user@example.com");
```

Output:

```
true
```

মনে রাখবে: production-grade email validation সাধারণত regex দিয়ে পুরো email specification implement করে না।

79. Phone Number Pattern

Basic example:

```
const phoneRegex = /^01\d{9}$/;
```

Example:

```
phoneRegex.test("01712345678");
```

Output:

```
true
```

এটা একটি simple application-specific pattern; সব possible phone-number format cover করে না।

80. Username Validation

```
const usernameRegex =  
  /^[a-zA-Z0-9_]{3,20}$/;
```

Rules:

- 3–20 characters
 - letters
 - numbers
 - underscore
-

81. Extract Numbers

```
const text =  
  "I have 10 apples and 25 oranges."  
  
const numbers =  
  text.match(/\d+/g);  
  
console.log(numbers);
```

Result:

```
["10", "25"]
```

82. Extract Hashtags

```
const text =  
  "Learning #JavaScript and #React";  
  
const hashtags =  
  text.match(/#[A-Za-z0-9_]+/g);  
  
console.log(hashtags);
```

83. Replace Extra Spaces

```
const text =  
  "Hello    world";  
  
const normalized =  
  text.replace(/\s+/g, " ");  
  
console.log(normalized);
```

Result:

```
Hello world
```

84. Remove Leading/Trailing Spaces

Regex দিয়ে:

```
const result =  
  text.replace(/^s+|s+$/g, "");
```

তবে সাধারণ use case-এ:

```
text.trim();
```

বেশি readable।

85. Regex-এর সবচেয়ে Important Building Blocks

```
Character
↓
Character class
↓
Quantifier
↓
Group
↓
Anchor
↓
Assertion
↓
Capture
```

Example:

```
/^(?<name>[A-Za-z]+\d{2,4})$/
```

এখানে অনেক concept একসাথে আছে।

86. Full Regex Breakdown Example

```
/^(?<username>[a-zA-Z][a-zA-Z0-9_]{2,15})$/
```

Breakdown:

```
^
↓
start

(?<username>...)
↓
named capture group

[a-zA-Z]
↓
first character must be letter
```

[a-zA-Z0-9_]

↓

allowed remaining characters

{2,15}

↓

2 to 15 additional characters

\$

↓

end

Total length:

3–16 characters

87. Regex Mental Model

একটি regex পড়ার সময় left-to-right parse করে:

^

↓

Where does it start?

character class

↓

What character is allowed?

quantifier

↓

How many?

group

↓

Which parts belong together?

assertion

↓

What condition must be true?

\$

↓

Where does it end?

88. Regex Debugging Strategy

Complex regex একবারে লিখবে না।

Step-by-step build করো।

ধরো number:

Step 1

```
/\d/
```

Step 2

```
/\d+/
```

Step 3

```
/^\d+$/
```

Step 4

```
/^\d{10,11}$/
```

প্রতিটি step test করো।

89. Common Mistake — Forgetting Anchors

এই regex:

```
/\d{5}/
```

একটি string-এর মধ্যে পাঁচটি digit থাকলেই match করতে পারে।

যদি পুরো string exactly 5 digits হতে হয়:

```
/^\d{5}$/
```

90. Common Mistake — `.` না Escape করা

Wrong:

```
/1.2/
```

এখানে `.` any character ।

যদি literal:

```
1.2
```

match করতে চাও:

```
/1\.2/
```

91. Common Mistake — Greedy `.*`

এই:

```
/.*/
```

অনেক বেশি text consume করতে পারে ।

যেখানে minimal matching দরকার:

```
/.*/?
```

92. Common Mistake — `\w` Unicode Word Character ভাবা

`\w` সাধারণভাবে ASCII-oriented word characters-এর জন্য ব্যবহৃত হয়:

```
A-Z  
a-z  
0-9  
—
```

International usernames/text-এর জন্য blindly `\w` ব্যবহার করা উচিত নয়।

Unicode property escapes consider করো:

```
/^\p{L}+$/u
```

93. Common Mistake — Regex দিয়ে Everything Validate করা

Regex powerful, কিন্তু সব validation regex দিয়ে করা উচিত নয়।

যেমন:

- complex date validity
- full email specification
- URL semantics
- mathematical expressions
- nested programming languages

কিছু ক্ষেত্রে parser বা dedicated API better।

94. Regex Performance

Complex regex backtracking-heavy হলে performance সমস্যা হতে পারে।

বিশেষ করে nested quantifiers:

```
/(a+)+$/
```

এর মতো patterns potentially expensive হতে পারে।

Large/untrusted input-এর উপর complex regex ব্যবহার করার সময় performance মাথায় রাখো।

95. Regex Security — ReDoS

Regular Expression Denial of Service (ReDoS) হলো এমন vulnerability যেখানে pathological regex input-এর কারণে excessive CPU time consume করতে পারে।

Dangerous pattern design এড়াও:

```
nested quantifiers  
ambiguous alternatives  
catastrophic backtracking
```

User-controlled large input-এর ক্ষেত্রে বিশেষভাবে সতর্ক হও।

96. Regex Object Properties

Useful properties:

```
const regex = /hello/gi;  
  
console.log(regex.source);  
console.log(regex.flags);  
console.log(regex.global);  
console.log(regex.ignoreCase);  
console.log(regex.multiline);  
console.log(regex.unicode);
```

97. RegExp.prototype.exec()

এই section-এর main topics-এর বাইরে হলেও `exec()` জানা useful।

```
const regex = /\d+/g;  
  
const match = regex.exec("Age: 25");
```

এটি detailed match information দেয়।

`matchAll()` modern repeated-match processing-এর জন্য অনেক ক্ষেত্রে easier।

98. Regex Methods Summary

```
test()  
↓  
Does it match?  
  
match()  
↓  
Give me matches  
  
matchAll()  
↓  
Give me all detailed matches  
  
replace()  
↓  
Replace matches  
  
search()  
↓  
Where is first match?  
  
split()  
↓  
Split using pattern
```

99. Practical Workflow

ধরো input validation করতে হবে।

Step 1 — Requirement define

```
Username:  
- 3–16 chars  
- starts with letter  
- letters/numbers/underscore
```

Step 2 — Build pattern

```
/^[A-Za-z][A-Za-z0-9_]{2,15}$/
```

Step 3 — Test

```
regex.test(username);
```

Step 4 — Edge cases

Test:

```
abc  
abc123  
a_b  
1abc  
ab  
very_long_username
```

100. Practice — Easy

Problem 1

এই string:

```
const text = "JavaScript is awesome";
```

Check করো "JavaScript" আছে কিনা।

Problem 2

একটি regex তৈরি করো যেটা শুধু digits accept করবে।

Valid:

12345

Invalid:

123a

Problem 3

একটি regex তৈরি করো যেটা exactly 5 digits match করবে।

Hint:

```
^
$
\d
{5}
```

101. Practice — Medium

Problem 4

এই string থেকে সব numbers বের করো:

```
const text =
  "I bought 5 books and 12 pens for 300 taka.";
```

Expected:

```
["5", "12", "300"]
```

Problem 5

এই string:

```
const text =  
  "hello hello world world";
```

শুধু repeated consecutive word detect করো।

Hint:

```
capture group  
+  
backreference
```

Problem 6

একটি regex তৈরি করো যেটা basic username validate করবে:

Rules:

- starts with letter
- 3-16 characters
- letters/numbers/underscore

102. Practice — Advanced

Problem 7 — Password

Requirements:

- minimum 8
- uppercase
- lowercase
- digit

Lookahead ব্যবহার করে regex লিখো।

Problem 8 — Named Groups

এই date:

2026-09-03

থেকে named groups দিয়ে বের করো:

```
{  
  year: "2026",  
  month: "09",  
  day: "03"  
}
```

Problem 9 — Lookbehind

এই text:

Price: \$500

শুধু:

500

match করো।

Hint:

(?<=...)

103. Practice — Unicode

Problem 10

একটি Unicode-aware regex তৈরি করো যেটা letters accept করবে:

Hello
বাংলা
العربية
日本語

Hint:

`[p{L}]`

এবং:

`/u`

ব্যবহার করো।

104. Output Prediction

Question 1

```
console.log(  
  /hello/i.test("HELLO")  
);
```

Output?

Question 2

```
console.log(  
  "123 abc 456".match(/\d+/g)  
);
```

Output?

Question 3

```
console.log(  
  "cat cat".match(/(cat)\s+\1/)  
);
```

Match হবে?

Question 4

```
console.log(  
  /^\d+$/ .test("12345")  
);
```

Output?

Question 5

```
console.log(  
  /( ?<=\$)\d+/.exec("$500")?.[0]  
);
```

Output?

105. Interview Questions

Beginner

1. Regular Expression কী?
2. Regex literal কী?
3. `RegExp` constructor কী?
4. Regex literal এবং constructor-এর difference কী?
5. Regex flag কী?
6. `g` এবং `i` flag কী?
7. `test()` কী return করে?

8. `match()` কী করে?
 9. `search()` এবং `indexOf()` -এর difference কী?
 10. `replace()` কীভাবে regex-এর সাথে কাজ করে?
-

Intermediate

1. `match()` এবং `matchAll()` -এর difference কী?
 2. Character class কী?
 3. `\d`, `\w`, `\s` কী?
 4. `*`, `+`, `?`, `{n,m}` কী?
 5. Greedy এবং lazy quantifier কী?
 6. Capturing group কী?
 7. Non-capturing group কী?
 8. Alternation `|` কী?
 9. Backreference কী?
 10. `^` এবং `$` কী?
-

Advanced

1. Lookahead কী?
 2. Lookbehind কী?
 3. Positive এবং negative lookahead-এর difference কী?
 4. Named capture group কী?
 5. Named backreference কী?
 6. `/u` flag কেন গুরুত্বপূর্ণ?
 7. Unicode property escape কী?
 8. `\w` কেন multilingual text-এর জন্য যথেষ্ট নাও হতে পারে?
 9. ReDoS কী?
 10. Regex কখন ব্যবহার না করাই ভালো?
-

106. Quick Revision Cheat Sheet

Creation

```
/hello/
```

```
new RegExp("hello");
```

Flags

```
g → global  
i → case-insensitive  
m → multiline  
s → dotAll  
u → Unicode  
y → sticky  
d → match indices
```

Methods

```
regex.test(text);
```

```
text.match(regex);
```

```
text.matchAll(regex);
```

```
text.replace(regex, replacement);
```

```
text.search(regex);
```

```
text.split(regex);
```

Character Classes

```
[abc]    → a/b/c  
[^abc]   → not a/b/c  
[a-z]    → lowercase  
[A-Z]    → uppercase  
[0-9]    → digits  
\d       → digit  
\D       → non-digit  
\w       → ASCII word char  
\W       → non-word
```

<code>\s</code>	→ whitespace
<code>\S</code>	→ non-whitespace

Quantifiers

<code>*</code>	→ 0+
<code>+</code>	→ 1+
<code>?</code>	→ 0 or 1
<code>{n}</code>	→ exactly n
<code>{n,}</code>	→ n+
<code>{n,m}</code>	→ n to m

Anchors

<code>^</code>	→ start
<code>\$</code>	→ end
<code>\b</code>	→ word boundary

Groups

<code>(...)</code>	→ capturing
<code>(?:...)</code>	→ non-capturing
<code>(?<name>...)</code>	→ named capture

Assertions

<code>(?=...)</code>	→ positive lookahead
<code>(?!...)</code>	→ negative lookahead
<code>(?<=...)</code>	→ positive lookbehind
<code>(?<!=...)</code>	→ negative lookbehind

Backreferences

\1
\2

Named:

\k<name>

Unicode

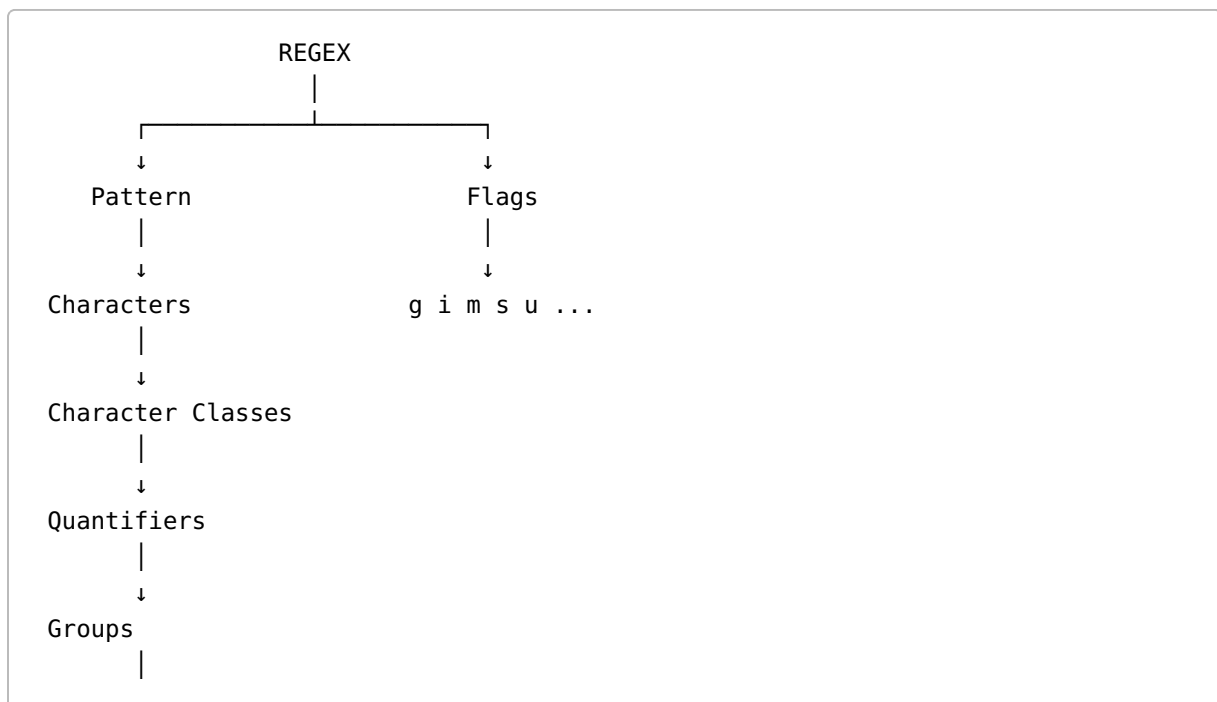
/\p{L}+/u

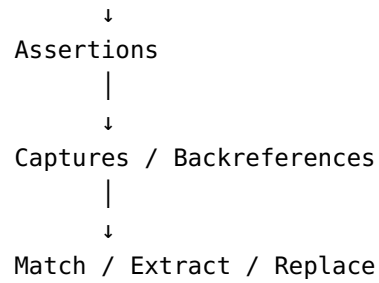
/\p{N}+/u



Final Mental Model

Regex-কে একটি **pattern language** হিসেবে ভাবো:





একটি complex regex সাধারণত ছোট ছোট building block দিয়ে তৈরি হয়:

```
Character
+
Class
+
Quantifier
+
Group
+
Anchor
+
Assertion
↓
Complete Pattern
```

Most Important Things to Master

প্রথমে এগুলো strong করো:

1. /pattern/flags
2. test()
3. match()
4. matchAll()
5. replace()
6. search()
7. split()
8. Character classes
9. \d \w \s
10. Quantifiers
11. ^ \$
12. Groups
13. Capturing groups
14. Non-capturing groups

- 15. Alternation
- 16. Greedy vs lazy
- 17. Lookahead
- 18. Lookbehind
- 19. Named groups
- 20. Backreferences
- 21. /u
- 22. Unicode property escapes

সবশেষে মনে রাখো:

Regex মুখস্থ করার জিনিস কম, **pattern build** করার **skill** বেশি।

একটা বড় **regex** দেখে ভয় না পেয়ে এটাকে ভেঙে পড়ো:

Where does it start?
↓
What characters are allowed?
↓
How many?
↓
What should be grouped?
↓
What condition must exist?
↓
Where does it end?

এই **mindset** develop করতে পারলে **complex regex**-ও ধীরে ধীরে পড়তে ও লিখতে পারবে।